

Relazione - Versione 4: Planner

Introduzione

La Versione 4 introduce il Planner.

Nella versione precedente, il progetto aveva introdotto il Generic Parser. L'agente era già capace di selezionare un tool, estrarre gli argomenti richiesti, eseguire il tool selezionato e salvare l'interazione in memoria.

La Versione 4 mantiene questa struttura, ma aggiunge un nuovo passaggio prima del parsing e dell'esecuzione: l'agente ora crea un semplice piano di esecuzione.

Anche questa versione rimane rule-based. Non viene usata una LLM e non viene introdotto nessun framework esterno.

Obiettivo della Versione 4

L'obiettivo principale della Versione 4 è introdurre il planning.

L'agente non dovrebbe eseguire subito dopo aver selezionato un tool. Dovrebbe prima creare un piccolo piano che descrive cosa sta per fare.

Il piano contiene tre informazioni:

- il tool selezionato;
- la ragione per cui quel tool viene usato;
- gli argomenti richiesti dal tool.

L'idea importante e semplice: l'agente inizia a ragionare prima di agire.

Cosa cambia rispetto alla Versione 3

La Versione 3 era concentrata sul Parser. Il Parser era diventato generico e basato sui parametri.

La Versione 4 non modifica il Parser come concetto principale. Il Parser continua a leggere i parametri richiesti dal tool selezionato ed estrae gli argomenti necessari.

Il nuovo componente è il Planner.

Il flusso passa da:

User Request -> Router -> Generic Parser -> Executor -> Memory

a:

User Request -> Router -> Planner -> Generic Parser -> Executor -> Memory

Il Router continua a selezionare il tool più adatto. Il Planner costruisce poi un piccolo piano di esecuzione intorno al tool selezionato.

Nuovo concetto: Planner

Il Planner è un nuovo componente che crea un piano prima che l'agente faccia parsing degli argomenti ed esegua un tool.

In questa versione, il Planner è semplice e rule-based. Usa il Router esistente per scegliere il tool più adatto.

Se viene trovato un tool adatto, il Planner restituisce un piano con il tool selezionato, la ragione per usarlo e i parametri richiesti dal tool.

Esempio:

```
{
  "tool": "addition",
  "reason": "Add two numbers.",
  "needs_arguments": ["a", "b"]
}
```

Questo piano viene poi salvato nello stato dell'agente.

Richieste non supportate

Il Planner gestisce anche le richieste non supportate.

Per esempio, la richiesta:

```
agent("What is the weather in Rome?")
```

non corrisponde a nessun tool disponibile.

In questo caso, il Planner restituisce:

```
{
  "tool": None,
  "reason": "No suitable tool was found for the request.",
  "needs_arguments": []
}
```

L'Agent restituisce poi un messaggio di fallback sicuro:

```
"I cannot handle this request yet."
```

Questo è importante perché l'agente deve fallire in modo chiaro e sicuro quando una richiesta non rientra nelle sue capacità attuali.

Flusso completo della Versione 4

Il flusso completo della Versione 4 è:

```
User Request -> Router -> Planner -> Generic Parser -> Executor -> Memory
↓
Output
```

Le responsabilità principali sono ora ancora più chiare:

```
Router = sceglie il tool
```

Planner = crea il piano di esecuzione
Parser = estrae gli argomenti
Executor = valida ed esegue
Agent = coordina
Memory = salva lo stato

L'Agent rimane il coordinatore. Ora coordina planning, parsing, esecuzione e salvataggio in memoria.

Stato dell'Agent nella Versione 4

Lo stato dell'agente ora contiene una nuova chiave: plan.

Uno stato tipico e questo:

```
{
  "request": "What is 2 + 3?",
  "plan": {
    "tool": "addition",
    "reason": "Add two numbers.",
    "needs_arguments": ["a", "b"]
  },
  "action": "addition",
  "arguments": {"a": 2, "b": 3},
  "result": 5
}
```

Questo rende piu facile osservare il comportamento interno dell'agente.

Test principali

I test verificano che l'agente continui a funzionare dopo l'aggiunta del Planner.

I casi principali sono:

```
agent("What is 2 + 3?")
agent("Hello, what is 2 + 3?")
agent("Hello, my name is luca.")
agent("What is the weather in Rome?")
memory
```

Questi test verificano che:

- il Router continui a dare priorit  all'intento principale;
- il Planner crei un piano prima dell'esecuzione;
- il Parser continui a estrarre gli argomenti corretti;
- l'Executor continui a validare ed eseguire il tool selezionato;
- le richieste non supportate vengano gestite senza tool selezionato;
- la Memory salvi il piano insieme al resto dello stato.

Cosa abbiamo imparato nella Versione 4

La Versione 4 insegna un principio importante nella progettazione degli agenti:

un agente dovrebbe pianificare prima di agire

Il Planner e ancora semplice, ma introduce l'idea che un agente possa creare una rappresentazione intermedia prima dell'esecuzione.

Questo rende l'architettura piu facile da estendere in futuro, soprattutto quando il progetto andra verso planner piu avanzati e agenti multi-step.

Limiti della Versione 4

La Versione 4 migliora l'architettura, ma alcuni limiti rimangono:

- il Router e ancora a regole;
- il Planner e ancora a regole;
- il Parser usa ancora regole manuali di estrazione;
- la memoria e ancora una semplice lista;
- non c'e ancora un Tool Registry;
- non c'e ancora una LLM;
- l'agente esegue ancora una sola azione alla volta.

Questi limiti sono normali e verranno affrontati gradualmente nelle versioni successive.

Conclusione

La Versione 4 e una naturale evoluzione della Versione 3.

Il Generic Parser rimane presente, ma il nuovo focus e il Planner.

L'agente ora crea un piccolo piano di esecuzione prima di fare parsing degli argomenti ed eseguire il tool selezionato.

Questo rende l'agente piu trasparente e prepara il progetto a comportamenti piu avanzati.

Il prossimo passo sara la Versione 5, dove il progetto introdurra il Memory Manager.